

Tabla de Contenido

1	DESCRIPCIÓN GENERAL DEL PROYECTO	6
2	PRINCIPIOS Y OBJETIVOS	6
3	ARQUITECTURA DE ALTO NIVEL	7
3.1	Flujo Típico:	7
4	MÓDULOS PRINCIPALES (VISIÓN POR CARPETAS)	7
5	QUÉ RESUELVE	9
6	CASOS DE USO TÍPICOS	10
7	DECISIONES DE DISEÑO CLAVE	10
8	CONVENCIONES Y ESTRUCTURA DE SOLUCIÓN.....	10
9	CÓMO EMPEZAR (ALTO NIVEL).....	10
9.1	Requisitos Sugeridos:.....	11
10	AUDIENCIA	11
11	FRAMEWORK.DDD.....	11
11.1	Introducción	11
11.2	Qué Resuelve	11
11.3	Qué Queda Fuera	11
11.4	Componentes principales y tecnologías.....	12
11.4.1	Componentes	12
11.4.2	Tecnologías y dependencias clave	12
11.5	Ejemplos De Uso Y Casos De Uso	13
11.5.1	Casos De Uso Típicos	14
11.5.2	Patrón de flujo (escritura).....	14
11.6	Ventajas De La Librería	15
12	FRAMEWORK.EDA.....	15
12.1	Introducción	15
12.2	Objetivo	16
12.3	Queda Fuera De Alcance	16
12.4	Componentes Principales Y Tecnologías.....	16
12.4.1	Componentes	16
12.4.2	Tecnologías y dependencias	17
12.5	Ejemplos De Uso Y Casos De Uso	17

12.6	Casos De Uso Típicos	19
12.7	Buenas prácticas operativas.....	19
12.8	Ventajas De La Librería	19
13	FRAMEWORK.CQRS.....	20
13.1	Introducción	20
13.2	Qué Resuelve	20
13.3	Qué Queda Fuera	20
13.4	Componentes principales y tecnologías.....	20
13.4.1	Componentes	20
13.4.2	Tecnologías y dependencias	21
13.5	Ejemplos De Uso Y Casos De Uso	21
13.6	Casos De Uso Típicos	24
13.7	Buenas Prácticas	24
13.8	Ventajas De La Librería	24
14	FRAMEWORK.FILESTORAGE.....	25
14.1	Introducción	25
14.2	Componentes Principales Y Tecnologías.....	25
14.2.1	Componentes (Ports).....	25
14.2.2	Adaptadores.....	26
14.2.3	Configuración.....	26
14.2.4	Dependencias y tecnologías	26
14.3	Ejemplos De Uso Y Casos De Uso	27
14.4	Casos De Uso Típicos	28
14.5	Buenas Prácticas	29
14.6	Ventajas De La Librería	29
15	FRAMEWORK.ENCRYPTOR.....	30
15.1	Introducción	30
15.2	Componentes Principales Y Tecnologías.....	30
15.2.1	Componentes	30
15.2.2	Configuración y claves.....	30
15.2.3	Dependencias y tecnologías	31
15.2.4	Buenas prácticas de configuración	31

15.3	Ejemplos De Uso Y Casos De Uso	31
15.4	Casos De Uso Típicos	32
15.5	Recomendaciones Operativas	32
15.6	Ventajas de la librería	32
16	FRAMEWORK.VALIDATIONS	33
16.1	Introducción	33
16.2	Objetivo	33
16.3	Componentes Principales Y Tecnologías	33
16.3.1	Componentes	33
16.3.2	Tecnologías y dependencias	33
16.3.3	Convenciones	34
16.4	Ejemplos de uso y casos de uso	34
16.5	Casos De Uso Típicos	35
16.6	Buenas Prácticas	36
16.7	Ventajas De La Librería	36
17	FRAMEWORK.CACHE	36
17.1	Introducción	36
17.2	Componentes Principales Y Tecnologías	37
17.2.1	Componentes	37
17.2.2	Tecnologías y dependencias	37
17.2.3	Convenciones	37
17.3	Ejemplos De Uso Y Casos De Uso	37
17.4	Casos De Uso Típicos	39
17.5	Buenas Prácticas	39
17.6	Ventajas De La Librería	39
18	FRAMEWORK.REPOSITORY	40
18.1	Introducción	40
18.2	Componentes Principales Y Tecnologías	40
18.2.1	Componentes	40
18.2.2	Tecnologías y dependencias	41
18.2.3	Convenciones	41
18.3	Ejemplos De Uso Y Casos De Uso	41

18.4	Casos de uso típicos	42
18.5	Buenas prácticas	42
18.6	Ventajas De La Librería	42
19	FRAMEWORK.MESSAGEBUS	43
19.1	introducción	43
19.2	Componentes Principales Y Tecnologías.....	43
19.2.1	Componentes	43
19.2.2	Tecnologías y dependencias	45
19.3	Ejemplos De Uso Y Casos De Uso	45
19.4	Flujo de suscripción con mediador	46
19.5	Casos De Uso Típicos.....	47
19.6	Buenas Prácticas	47
19.7	Ventajas De La Librería	47
20	FRAMEWORK.SCAFFOLDING	48
20.1	Introducción	48
20.2	Objetivo	48
20.3	Qué Resuelve	48
20.4	Componentes principales y tecnologías.....	48
20.4.1	Componentes	48
20.4.2	Tecnologías y dependencias	49
20.5	Ejemplos De Uso Y Casos De Uso	49
20.6	Casos De Uso Típicos.....	51
20.7	Buenas Prácticas	51
20.8	Ventajas De La Librería	51
21	FRAMEWORK.TRANSACTION	52
21.1	Introducción	52
21.2	Componentes Principales Y Tecnologías.....	52
21.2.1	Componentes	52
21.2.2	Tecnologías y dependencias	53
21.3	Ejemplos De Uso Y Casos De Uso	53
21.4	Caso de uso: Integración con MessageBus/Eda.....	55
21.5	Ventajas De La Librería	55

22	FRAMEWORK.LOGGER.....	56
22.1	Introducción	56
22.2	Componentes principales y tecnologías.....	56
22.2.1	Componente	56
22.2.2	Tecnologías y dependencias	56
22.2.3	Convenciones	56
22.3	Ejemplos de uso y casos de uso.....	57
22.4	Casos De Uso Típicos	57
22.5	Buenas Prácticas	57
22.6	Ventajas De La Librería	58
23	FRAMEWORK.CONFIGURATION	58
23.1	Introducción	58
23.2	Objetivo	58
23.3	Componentes principales y tecnologías.....	58
23.3.1	Componente	58
23.3.2	Tecnologías y dependencias	58
23.3.3	Convenciones	59
23.4	Ejemplos De Uso Y Casos De Uso	59
23.5	Casos De Uso Típicos	60
23.6	Buenas Prácticas	60
23.7	Ventajas De La Librería	60
24	FRAMEWORK.UTILS	60
24.1	Introducción	60
24.2	Componentes Principales Y Tecnologías.....	60
24.2.1	Tecnologías y dependencias	61
24.2.2	Convenciones	61
24.3	Ejemplos de uso y casos de uso.....	61
24.4	Casos de uso típicos	61
24.5	Buenas prácticas	62
24.6	Ventajas de la librería	62
25	CONTROL DE ACCESO.....	62
26	CONTROL DE VERSIONES	62

1 DESCRIPCIÓN GENERAL DEL PROYECTO

Este repositorio es un framework de referencia en .NET 8 diseñado para construir aplicaciones empresariales basadas en patrones modernos: CQRS (Command Query Responsibility Segregation), EDA (Event-Driven Architecture), DDD (Domain-Driven Design), mensajería asíncrona y técnicas de resiliencia como Outbox. Proporciona:

- Abstracciones y contratos (puertos) para desacoplar el dominio de la infraestructura.
- Implementaciones adaptables (adapters) para tecnologías comunes: RabbitMQ, Redis, EF Core, MongoDB, proveedores de logs, almacenamiento de archivos, cifrado y configuración.
- Capas y utilidades transversales: validaciones, logging, transacciones, caché, configuración, scaffolding.
- Proyectos de ejemplo end-to-end (Command API, Query API, dominio y aplicación) para acelerar el arranque y servir como guía.

En esencia, el framework permite montar soluciones escalables, mantenibles y alineadas con prácticas de arquitectura limpia y hexagonal, cuidando la separación entre dominio, aplicación e infraestructura.

2 PRINCIPIOS Y OBJETIVOS

- Separación de responsabilidades (CQRS): comandos para mutaciones del estado y consultas independientes para lecturas optimizadas.
- Event-Driven Architecture: publicación y suscripción de eventos para integrar bounded contexts y servicios.
- DDD-first: el dominio manda, la infraestructura se pluga por contratos.
- Port/Adapter: interfaces (ports) acopladas al dominio; adapters reemplazables para proveedores.
- Resiliencia y consistencia: patrón Outbox, transacciones y reintentos controlados.
- Observabilidad y gobernanza: logging unificado, configuración tipada y validaciones.
- Productividad: scaffolding y ejemplos listos para usar.

3 ARQUITECTURA DE ALTO NIVEL

- Capa de Dominio (DDD)
 - Entidades, agregados, eventos de dominio, invariantes y lógica de negocio.
- Capa de Aplicación
 - Commands/Queries + Handlers, orquestación, validaciones y transacciones.
- Mensajería
 - Bus de eventos/comandos, publicación, suscripción y jobs auxiliares (Outbox, consumidores).
- Persistencia
 - Repositorios abstraídos con implementaciones para EF Core y MongoDB.
- Infraestructura transversal
 - Cache (Redis), configuración, logging, cifrado, almacenamiento de archivos.
- APIs
 - Endpoints para expone Command API (escritura) y Query API (lectura).

3.1 Flujo Típico:

- Escritura: API de comandos -> Handler -> Dominio -> Persistencia (EF/Mongo) -> Outbox -> Publicación asíncrona.
- Lectura: API de consultas -> Handler -> Read Model (proyección optimizada).
- Integración: Suscriptores de eventos -> Handlers asíncronos -> Actualización de proyecciones/acciones de integración.

4 MÓDULOS PRINCIPALES (VISIÓN POR CARPETAS)

- Framework.Cqrs
 - Contratos y utilidades para Command/Query, Handlers y buses.
- Framework.Cqrs.Api
 - Infraestructura para exponer endpoints CQRS (p. ej., mediador, middlewares comunes).
- Framework.Cqrs.CommandBus
 - Abstracción del bus de comandos (si aplica en escenarios distribuidos).

- Framework.Eda
 - Contratos para eventos, publicación/suscripción y orquestación.
- Framework.Eda.Outbox.Job
 - Job para leer la Outbox y publicar eventos de forma confiable.
- Framework.Ddd.DomainEvents
 - Manejo de eventos de dominio dentro del contexto DDD.
- Framework.Ddd.EventSourcing
 - Soporte para Event Sourcing (si se habilita en el dominio).
- Framework.MessageBus
 - Puerto de mensajería (publish/subscribe).
- Framework.MessageBus.Rabbitmq
 - Adapter para RabbitMQ.
- Framework.MessageBus.Subscribe.Job
 - Proceso/Job para suscripción y procesamiento de mensajes.
- Framework.Repository
 - Puerto y abstracciones de repositorios.
- Framework.Repository.EntityFramework
 - Adapter con EF Core.
- Framework.Repository.Mongodb
 - Adapter para MongoDB.
- Framework.Cache
 - Abstracción de caché.
- Framework.Cache.Reddis
 - Adapter para Redis (nota: nombre de carpeta “Reddis”).
- Framework.Logger
 - Abstracción de logging.
- Framework.Logger.NetLogger
 - Integración con el logger de .NET (ILogger).
- Framework.Configuration

- Abstracciones de configuración.
- Framework.Configuration.NetConfiguration
 - Adapter para configuración .NET (IConfiguration, options, etc.).
- Framework.Transaction
 - Manejo de transacciones.
- Framework.Transaction.Extensions
 - Extensiones para integración transaccional (p. ej., unit of work, ambient context).
- Framework.Validations
 - Validaciones transversales (p. ej., FluentValidation u otra estrategia).
- Framework.Encryptor.Port / Framework.Encryptor.Adapter
 - Puerto y adapter para cifrado/descifrado.
- Framework.FileStorage.Port / Framework.FileStorage.Adapters
 - Puerto y adapters para almacenamiento de archivos (S3, filesystem, etc.).
- Framework.Scaffolding
 - Utilidades para generación de código y convenciones base.
- Framework.Scaffolding.Bolt
 - Herramientas de scaffolding acelerado.
- Framework.Scaffolding.ServiceCollection
 - Extensiones de DI para registrar módulos.
- Framework.Scaffolding.Test.Api
 - API de prueba para validar el ensamblaje de módulos.
- Framework.Utils
 - Utilidades generales (helpers, extensiones).

5 QUÉ RESUELVE

- Proveer una base opinada y extensible para microservicios o bounded contexts orientados a eventos.
- Reducir tiempo de arranque del proyecto con ejemplos y scaffolding.

- Homogeneizar cross-cutting concerns (logging, config, validaciones, transacciones, caché).
- Aislar el dominio de la infraestructura con contratos claros.
- Facilitar consistencia eventual y comunicación asíncrona confiable (Outbox + MessageBus).

6 CASOS DE USO TÍPICOS

- Sistemas con alto volumen de escritura y necesidad de escalar lecturas con proyecciones.
- Integración entre servicios mediante eventos de integración.
- Dominios complejos que requieren invariantes fuertes y modelado táctico (DDD).
- Necesidad de múltiples tecnologías de persistencia (EF/Mongo) y caché (Redis).
- Requisitos de observabilidad, gobernanza y trazabilidad de eventos.

7 DECISIONES DE DISEÑO CLAVE

- CQRS: desacopla lectura y escritura para optimizar cada camino.
- Outbox: garantiza publicación de eventos tras commit de la transacción.
- Ports/Adapters: intercambiabilidad de proveedores (RabbitMQ, Redis, EF, Mongo).
- Jobs dedicados: separación de responsabilidades para publicación y suscripción.
- Ejemplos concretos: guía estructura, nombres y mejores prácticas.

8 CONVENCIONES Y ESTRUCTURA DE SOLUCIÓN

- Solución .NET 8 (Framework.sln) con proyectos por capa/módulo.
- Nombres por responsabilidad: Port/Adapter, Command/Query, Handlers, Api, Jobs.
- Inyección de dependencias mediante extensiones en ServiceCollection.
- Eventos diferenciados: de dominio (internos) vs. de integración (entre servicios).
- Read Models desacoplados, optimizados para consultas.

9 CÓMO EMPEZAR (ALTO NIVEL)

- Revisa los proyectos Example.* para entender el flujo end-to-end.

- Define tu dominio en una nueva carpeta/solución “YourContext.Domain”.
- Implementa Application con Commands/Queries/Handlers.
- Selecciona adaptadores: EF/Mongo, RabbitMQ, Redis, y configura en tu arranque.
- Expone APIs: Command API para escrituras y Query API para lecturas.
- Despliega y ejecuta jobs: Outbox (publicación) y Subscribe (consumidores).
- Agrega validaciones, logging y configuración tipada según tus necesidades.

9.1 Requisitos Sugeridos:

- .NET 8 SDK.
- RabbitMQ y Redis si usas sus adapters.
- SQL o MongoDB según tu elección de repositorio.

10 AUDIENCIA

- Equipos de backend .NET que buscan un punto de partida sólido para arquitecturas DDD, CQRS y EDA.
- Arquitectos y desarrolladores que necesiten modularidad, escalabilidad y buenas prácticas por defecto.

11 FRAMEWORK.DDD

11.1 Introducción

Framework.Ddd ofrece los bloques esenciales para aplicar Domain-Driven Design con Event Sourcing y eventos de dominio. Su objetivo es modelar agregados que evolucionan por eventos, persistir su historial de cambios y rehidratar su estado de manera determinista, integrándose con mensajería (EDA) y repositorios sin acoplar el dominio a la infraestructura.

11.2 Qué Resuelve

- Estado como una secuencia de hechos (eventos) y no como “último snapshot”.
- Publicación de eventos tras la persistencia para integraciones asíncronas.
- Rehidratación consistente del agregado aplicando el historial en orden.

11.3 Qué Queda Fuera

- Idempotencia y reintentos de consumidores externos (se recomienda Outbox + políticas en el bus).

- Snapshots y particionamiento (pueden añadirse como extensiones).
- Control de concurrencia a nivel de almacenamiento (depende del adapter de repositorio).

11.4 COMPONENTES PRINCIPALES Y TECNOLOGÍAS

11.4.1 Componentes

- DomainEvent
 - Contrato base de eventos de dominio. Incluye AggregateId, Version, GenerateDate e Id.
 - Compatible con la capa EDA del framework para su publicación.
- AggregateRoot
 - Clase base para agregados event-sourced.
 - Permite registrar “apply” por tipo de evento, generar eventos (RaiseEvent), llevar cambios no confirmados, rehidratar desde historial y mantener Version.
- IRepository
 - Abstracción para guardar y recuperar agregados por Id, sin acoplarse a un storage específico.
- Repository

Implementación genérica que:

 - Serializa eventos y los agrega a un stream (AggregateStore con EventStore).
 - Publica los eventos a la capa EDA después de persistirlos.
 - Limpia la cola de cambios (MarkChangesAsCommitted).
- EventStore y AggregateStore
 - Representación persistente del stream de eventos por agregado (datos serializados, tipo, versión, fecha).
- EventSourcingException
 - Señala errores de aplicación de eventos (por ejemplo, “apply” inexistente).

11.4.2 Tecnologías y dependencias clave

- **.NET 8:**

- Serialización con System.Text.Json (config del framework para compatibilidad de tipos).
- **Integración con:**
 - Framework.Repository (adapters EF Core, MongoDB, etc.) para persistencia del stream.
 - Framework.Eda (publisher) para publicación de eventos a la mensajería.
 - Inyección de dependencias mediante ServiceCollection (extensiones del framework).

11.5 Ejemplos De Uso Y Casos De Uso

Ejemplo mínimo de modelado y uso

```
// C#
public class ClienteCreado : DomainEvent
{
    public string Nombre { get; set; } = default!;
}

public class Cliente : AggregateRoot
{
    public string Nombre { get; private set; } = default!;

    public Cliente()
    {
        // Aplica el estado cuando ocurra el evento
        AddApply<ClienteCreado>(e => Nombre = e.Nombre);
    }

    public static Cliente Crear(string nombre)
    {
        var cliente = new Cliente();
        // Dispara el evento de dominio; actualiza estado y
        versiona
        cliente.RaiseEvent(new ClienteCreado { Nombre = nombre });
        return cliente;
    }
}

public class CrearClienteHandler
{
    private readonly IRepository<Cliente> _repo;
    public CrearClienteHandler(IRepository<Cliente> repo) => _repo
    = repo;

    public async Task<Guid> HandleAsync(string nombre)
```

```
        {
            var cliente = Cliente.Crear(nombre);
            await _repo.SaveChangesAsync(); // Persiste el stream y publica el evento
            return cliente.Id;
        }
    }

    public class CambiarNombreCliente : DomainEvent
    {
        public string NuevoNombre { get; set; } = default!;
    }

    public partial class Cliente
    {
        public void CambiarNombre(string nuevo)
        {
            // Aquí puedes validar invariantes antes de emitir el evento
            RaiseEvent(new CambiarNombreCliente { NuevoNombre = nuevo });
        }

        public Cliente() // registra también el nuevo apply
        {
            AddApply<CambiarNombreCliente>(e => Nombre = e.NuevoNombre);
        }
    }
}
```

11.5.1 Casos De Uso Típicos

- Trazabilidad total de cambios: auditoría completa a partir del stream de eventos.
- Dominios con reglas evolutivas: facilita versionar eventos y rehidratar con compatibilidad.
- Lecturas escalables: alimenta proyecciones y read models mediante publicación de eventos.
- Integraciones entre bounded contexts: eventos de dominio como fuente de eventos de integración.

11.5.2 Patrón de flujo (escritura)

- Handler de aplicación invoca métodos del agregado → el agregado emite eventos → Repository guarda y publica → consumidores externos procesan y proyectan.
- Patrón de flujo (rehidratación)

- Repository recupera stream por AggregateId → AggregateRoot aplica cada evento en orden → estado actual consistente y Version actualizado.

11.6 Ventajas De La Librería

- Desacoplamiento real
 - El dominio no depende de proveedores de almacenamiento ni de mensajería; se integra por puertos.
- Auditoría y transparencia
 - Cada cambio relevante queda registrado como evento con metadatos (Id, versión, fecha).
- Consistencia y resiliencia
 - Publicación de eventos tras persistirlos; combinable con Outbox para garantizar entrega.
- Escalabilidad de lecturas
 - Facilita proyecciones y modelos de lectura optimizados (CQRS).
- Evolución controlada
 - Versionado de eventos y posibilidad de transformar durante la rehidratación.
- Testabilidad
 - Los tests pueden verificar que, dadas ciertas acciones del agregado, se generan eventos específicos.
- Extensibilidad
 - Puedes añadir control de concurrencia optimista, snapshots, políticas de serialización y nuevos adapters de repositorio/bus sin tocar el dominio.

12 FRAMEWORK.EDA

12.1 Introducción

Framework.EDA aporta los bloques de arquitectura orientada a eventos (EDA) para aplicaciones .NET, asegurando:

- Un contrato unificado de evento de integración.
- Publicación de eventos desacoplada del bus de mensajería mediante Outbox.

- Ejecución inmediata de suscriptores in-process (cuando aplica) y publicación asíncrona para integraciones fuera del proceso.
- Propagación de metadatos transaccionales y de correlación.

12.2 Objetivo

Permitir que cualquier módulo de la aplicación publique eventos de manera confiable y que los consumidores los procesen en forma desacoplada, evitando dependencia directa del bus y reduciendo la posibilidad de perder mensajes.

12.3 Queda Fuera De Alcance

- Políticas de reintentos y DLQ del broker (las maneja el adapter del MessageBus).
- Transformaciones avanzadas de esquemas/versionado de eventos (se recomienda tratarlas en una capa de compatibilidad).
- Programación/scheduling del job de Outbox (lo integras con tu herramienta de jobs preferida).

12.4 Componentes Principales Y Tecnologías

12.4.1 Componentes

- Event
 - Contrato base de evento de integración. Incluye: Id, PublishDate y un objeto Information para trazabilidad/correlación proveniente del contexto transaccional.
- IEventPublisher
 - Puerto para publicar eventos desde la aplicación (command handlers, domain services, etc.). Si el evento está marcado para publicación externa, se persiste en Outbox; si existen handlers in-process, se ejecutan inmediatamente dentro del alcance transaccional.
- EventPublisher: Implementación que
 - Enriquece el evento con metadatos transaccionales (Information).
 - Invoca handlers in-process compatibles con el tipo de evento.
 - Si el evento está marcado para publicación externa (vía atributos), serializa y guarda en Outbox para publicación posterior.
- Outbox (EventOutboxModel)
 - Modelo de almacenamiento confiable para eventos pendientes de publicar en el broker (Topic, RoutingKey, Body, EventName, PublisherDate).

- IOutboxEventPublisher y OutboxEventPublisher
 - Servicio de “drenado” de Outbox: toma eventos pendientes y los publica al broker; al finalizar, elimina la entrada persistida.
- IEventHandlerContrato de suscriptores in-process (en la misma app) que reaccionan a eventos específicos.
 - Atributos de publicación/suscripción (a nivel de evento/handler)
- Permiten declarar si un evento debe ser publicado externamente (p. ej., asignando un nombre de mensaje/routing) y controlar el comportamiento de suscripción in-process.

12.4.2 Tecnologías y dependencias

.NET 8: Serialización con System.Text.Json (nomenclatura camelCase).

Integración con:

- Framework.Transaction para metadatos (Information) y resolución de dependencias dentro del alcance transaccional.
- Framework.Repository para persistir Outbox.
- Framework.MessageBus para la publicación efectiva hacia el broker (adapters como RabbitMQ).
- Inyección de dependencias con ServiceCollection.

12.5 Ejemplos De Uso Y Casos De Uso

Ejemplo 1: Publicar un evento y ejecutar handlers in-process

```
// C#
public class ClienteCreadoEvent : Event
{
    public Guid ClienteId { get; set; }
    public string Nombre { get; set; } = default!;
}

// Handler in-process
public class EnviarBienvenidaHandler :
    IEventHandler<ClienteCreadoEvent>
{
    public Task HandleAsync(ClienteCreadoEvent evt)
    {
        // Lógica interna: enviar email, auditar, etc.
        return Task.CompletedTask;
    }
}
```

```
// Emisión desde aplicación (p. ej., en un command handler)
public class CrearClienteAppService
{
    private readonly IEventPublisher _publisher;
    public CrearClienteAppService(IEventPublisher publisher) =>
        _publisher = publisher;

    public async Task EjecutarAsync(Guid id, string nombre)
    {
        // ... lógica de creación + persistencia transaccional ...
        await _publisher.PublishAsync(new ClienteCreadoEvent { Id =
Guid.NewGuid(), ClienteId = id, Nombre = nombre });
        // Los handlers in-process se ejecutan en este punto.
        // Si el evento está configurado para publicación externa,
        se guarda en Outbox.
    }
}
```

Ejemplo 2: Publicación externa con Outbox

```
// C#
// Marca el evento con la metadata necesaria (p. ej., nombre de
mensaje/routing mediante atributos)
// y emítelo como en el ejemplo anterior.

// Job/Worker que drena Outbox periódicamente
public class OutboxJob : BackgroundService
{
    private readonly IOutboxEventPublisher _outbox;
    public OutboxJob(IOutboxEventPublisher outbox) => _outbox =
outbox;

    protected override async Task ExecuteAsync(CancellationToken
stoppingToken)
    {
        while (!stoppingToken.IsCancellationRequested)
        {
            await _outbox.PublishAsync(); // Publica al broker y
limpia los registros procesados
            await Task.Delay(TimeSpan.FromSeconds(5),
stoppingToken);
        }
    }
}
```

12.6 Casos De Uso Típicos

- Integración entre bounded contexts y microservicios: publicar eventos de negocio fuera de la app sin acoplarse al broker.
- Proyecciones y read models: reaccionar in-process para mantener modelos de lectura locales (CQRS).
- Fiabilidad en la entrega: persistir el evento antes de publicarlo; un job lo enviará, incluso si el broker está caído temporalmente.
- Enriquecimiento de telemetría: propagar correlación y metadatos transaccionales en los eventos.

12.7 BUENAS PRÁCTICAS OPERATIVAS

- Ejecuta el job de Outbox con una cadencia acorde a tus SLA (o usa triggers de cambio).
- Mide y alerta ante acumulación de Outbox (backlog creciente).
- Los handlers in-process deben ser rápidos y, si son potencialmente lentos, delegar trabajo asíncrono.

12.8 Ventajas De La Librería

- Desacoplamiento del broker
 - La aplicación publica vía IEventPublisher sin conocer detalles del bus. Outbox abstrae la publicación efectiva.
- Confiabilidad (Outbox)
 - Garantiza que un evento persistido no se pierde; el job reintenta publicación hasta que el broker lo acepte.
- Consistencia con la transacción de negocio
 - Los eventos se generan con metadatos del contexto transaccional y se persisten de forma coherente con el cambio de estado.
- Doble vía de reacción
 - Handlers in-process para side-effects internos inmediatos.
 - Publicación externa para integración entre servicios.
- Observabilidad y trazabilidad
 - Metadatos de correlación (Information) facilitan seguir un evento de extremo a extremo.
- Extensibilidad

- Puedes cambiar el broker (RabbitMQ, etc.) sin tocar tus casos de uso.
- Puedes enriquecer serialización, esquema de eventos y políticas del job sin afectar el dominio.

13 FRAMEWORK.CQRS

13.1 INTRODUCCIÓN

Framework.Cqrs proporciona los bloques de construcción para aplicar el patrón Command Query Responsibility Segregation (CQRS) en .NET 8. Su objetivo es separar claramente:

- Escrituras (Commands) que mutan el estado del sistema.
- Lecturas (Queries) optimizadas para obtener datos.

El framework estandariza contratos de comandos y consultas, define interfaces de handlers, y ofrece un Mediator que resuelve y ejecuta el handler adecuado, propagando metadatos de correlación y contexto transaccional. Además, permite anotar comandos/consultas con atributos para nombres canónicos y reglas de autorización.

13.2 Qué Resuelve

- Separación de responsabilidades entre lectura y escritura.
- Enrutamiento de requests hacia su handler mediante un Mediator.
- Propagación de Information (correlación, tenant, usuario, etc.) con cada request.
- Nomenclatura y metadatos consistentes para auditar y monitorear.

13.3 Qué Queda Fuera

- Lógica de transporte (HTTP, mensajería, etc.); CQRS aquí es agnóstico al canal.
- Implementaciones concretas de autorización o validación; pueden integrarse como middlewares/pipelines alrededor del Mediator.
- Transaccionalidad de infra; se integra con el módulo de transacciones del framework.

13.4 COMPONENTES PRINCIPALES Y TECNOLOGÍAS

13.4.1 Componentes

- IRequest
 - Contrato base de cualquier request CQRS.

- Incluye RequestId y Information para trazabilidad, correlación y metadatos del contexto.
- ICommand / ICommand
 - Requests de escritura. Opcionalmente devuelven un resultado tipado.
 - Exponen CommandName y ResponseType (este último derivado del genérico).
- IQuery
 - Requests de lectura. Devuelven un resultado tipado.
 - Exponen QueryName y ResponseType.
- Handlers
 - ICommandHandler ICommandHandler
 - IQueryHandler<TQuery, TResponse>
 - Encapsulan la lógica de aplicación por caso de uso (unit of work, invariantes, orquestación).
- IMediator
 - Punto de entrada para ejecutar un request.
 - Expone MediateGenericAsync(TRequest, requestName?) y MediateAsync(object, requestName?).
 - Resuelve el handler apropiado según el tipo del request y lo ejecuta.
- Atributos
 - CqrsNameAttribute: asigna un nombre canónico a un comando/consulta (útil para logging, auditoría, roteo, contratos).
 - AuthAttribute: declara roles requeridos (permite integrar una política de autorización alrededor del Mediator).

13.4.2 Tecnologías y dependencias

.NET 8

- Integración con el módulo de transacciones: los IRequest llevan Information para el contexto transaccional y de correlación.
- Inyección de dependencias: resolución de handlers por tipo.

13.5 Ejemplos De Uso Y Casos De Uso

Ejemplo 1: Command sin retorno

```
// C#
using Framework.Cqrs.Concepts;
using Framework.Cqrs.Services;

public class ActivarUsuario : ICommand
{
    public Guid RequestId { get; set; } = Guid.NewGuid();
    public string? CommandName { get; set; } = "Usuarios.Activar";
    public Information? Information { get; set; }
    public Guid UsuarioId { get; set; }
}

public class ActivarUsuarioHandler :
    ICommandHandler<ActivarUsuario>
{
    public async Task HandleAsync(ActivarUsuario command)
    {
        // Lógica de activación + persistencia transaccional
        await Task.CompletedTask;
    }
}

// Uso con Mediator (p. ej., desde un endpoint)
public async Task EjecutarAsync(IMediator mediator, Guid usuarioId,
    Information info)
{
    await mediator.MediateGenericAsync(new ActivarUsuario {
        UsuarioId = usuarioId, Information = info });
}
```

Ejemplo 2: Command con retorno y Query

```
// C#
using Framework.Cqrs.Concepts;
using Framework.Cqrs.Services;

// Command con retorno
public class CrearUsuario : ICommand<Guid>
{
    public Guid RequestId { get; set; } = Guid.NewGuid();
    public string? CommandName { get; set; } = "Usuarios.Crear";
    public Information? Information { get; set; }
    public string Email { get; set; } = default!;
}

public class CrearUsuarioHandler : ICommandHandler<CrearUsuario,
```

```
Guid>
{
    public async Task<Guid> HandleAsync(CrearUsuario cmd)
    {
        // Persistir y devolver Id
        return await Task.FromResult(Guid.NewGuid());
    }
}

// Query tipada
public class ObtenerUsuario : IQuery<UsuarioDto>
{
    public Guid RequestId { get; set; } = Guid.NewGuid();
    public string? QueryName { get; set; } = "Usuarios.Obtener";
    public Information? Information { get; set; }
    public Guid UsuarioId { get; set; }
}

public record UsuarioDto(Guid Id, string Email);

public class ObtenerUsuarioHandler : IQueryHandler<ObtenerUsuario,
UsuarioDto>
{
    public Task<UsuarioDto> HandleAsync(ObtenerUsuario query)
    {
        // Lectura optimizada (read model)
        return Task.FromResult(new UsuarioDto(query.UsuarioId,
"test@dominio.com"));
    }
}

// Uso con Mediator
public async Task<Guid> CrearYLeerAsync(IMediator mediator, string
email, Information info)
{
    var id = (Guid)(await mediator.MediateGenericAsync(new
CrearUsuario { Email = email, Information = info }!));
    var dto = (UsuarioDto)(await mediator.MediateGenericAsync(new
ObtenerUsuario { UsuarioId = id, Information = info }!));
    return dto.Id;
}
```

13.6 Casos De Uso Típicos

- Escrituras con invariantes de dominio: encapsuladas en comandos, handlers y eventualmente agregados DDD.
- Lecturas escalables: queries que leen modelos de lectura o proyecciones específicas.
- Orquestación de procesos: handlers que coordinan repositorios, buses y servicios de dominio.
- Auditoría y trazabilidad: uso de RequestId, CommandName/QueryName e Information para logs y métricas.
- Autorización declarativa: AuthAttribute sobre comandos/consultas y un middleware/pipeline que valide roles.

13.7 Buenas Prácticas

- Un handler por tipo de request: responsabilidad única y testabilidad.
- Nombres canónicos (CqrsNameAttribute o propiedades Name) para observabilidad y contratos.
- Validaciones antes de ejecutar la lógica (pueden integrarse como pipeline/filters).
- No mezclar efectos de lectura y escritura en el mismo request.
- Propagar Information en todo el flujo (correlación).

13.8 Ventajas De La Librería

- Separación clara de responsabilidades
 - Commands para mutación; Queries para lectura. Facilita optimización y escalabilidad independiente.
- Desacoplamiento mediante Mediator
 - El llamador no conoce el handler concreto; resolución por tipo simplifica la arquitectura y los tests.
- Trazabilidad y gobernanza
 - RequestId, CommandName/QueryName e Information permiten correlación, auditoría y métricas consistentes.
- Extensibilidad por atributos
 - Nombres canónicos y requisitos de autorización declarativos, integrables en middlewares/pipelines.
- Testabilidad

- Handlers aislados; fácil mockear dependencias e inspeccionar efectos.
- Integración natural con DDD y EDA
 - Commands disparan lógica de dominio y eventos; Queries consumen proyecciones generadas por la capa EDA.

14 FRAMEWORK.FILESTORAGE

14.1 Introducción

Framework.FileStorage proporciona puertos (interfaces) y adaptadores para gestionar almacenamiento de archivos en aplicaciones .NET 8, manteniendo un acoplamiento mínimo con el proveedor subyacente. Soporta:

- Operaciones básicas de archivos: guardar, eliminar y descargar streams.
- Cálculo de tamaño y utilidades auxiliares.
- Cargas en chunks (bloques) para archivos grandes.
- Streaming parcial (rango de bytes), ideal para video/audio.
- Consulta de metadatos (propiedades del archivo).

El enfoque Port/Adapter permite cambiar de proveedor (por ejemplo, Azure Blob Storage) sin modificar la lógica de aplicación.

14.2 Componentes Principales Y Tecnologías

14.2.1 Componentes (Ports)

- IFileSystemPort
 - Guardar un archivo (byte[]) en un contenedor con un nombre.
 - Eliminar un archivo por nombre y contenedor.
 - Descargar un archivo a un Stream provisto.
 - Calcular tamaño en MB de un arreglo de bytes.
- IFileChunkSystemPort
 - Iniciar una carga en bloques (StartUpload).
 - Subir un bloque (UploadChunk).
 - Completar la carga consolidando los bloques (CompleteUpload).
 - Abortar la carga y limpiar bloques temporales (AbortUpload).

- Obtener un segmento del archivo para streaming (GetVideoChunk) con soporte de cabecera Range.
- Consultar propiedades/metadata (GetPropertiesFile).

14.2.2 Adaptadores

- BlobStorageAdapter
 - Implementa IFileSystemPort sobre Azure Blob Storage.
 - Crea contenedores si no existen; valida existencia al guardar; descarga a stream.
- BlobStorageFileChunkSystemAdapter
 - Implementa IFileChunkSystemPort sobre Azure Blob Storage.
 - Usa blobs por bloques (Block Blobs) para cargas multipartes.
 - Soporta Content-Range para streaming de medios.
 - Expone propiedades como ContentLength.

14.2.3 Configuración

- Requiere una cadena de conexión del proveedor de archivos (por ejemplo, Azure Blob Storage).
- Clave de configuración esperada (convención): filestorage.connectionstring.
- Se integra con el módulo de configuración del framework para obtenerla de forma tipada.

14.2.4 Dependencias y tecnologías

.NET 8

- Azure Storage Blobs (para el adapter incluido).
- Inyección de dependencias (ServiceCollection) para registrar puertos y adaptadores.
- Manejo de excepciones específico de almacenamiento (StorageException) para casos como “archivo ya existe”.

14.3 Ejemplos De Uso Y Casos De Uso

Ejemplo 1: Operaciones básicas de archivo

```
// C#
using Framework.FileStorage.Port;

public class ArchivosService
{
    private readonly IFileSystemPort _files;

    public ArchivosService(IFileSystemPort files) => _files =
files;

    public async Task<string> SubirAsync(byte[] contenido, string
nombre, string contenedor)
    {
        // Guarda el archivo y devuelve la ruta/URL relativa
        return await _files.SaveAsync(contenido, nombre,
contenedor);
    }

    public async Task DescargarAsync(string nombre, string
contenedor, Stream destino)
    {
        // Descarga el archivo al stream provisto (por ejemplo,
Response.Body)
        await _files.DownloadAsync(nombre, contenedor, destino);
    }

    public async Task EliminarAsync(string nombre, string
contenedor)
    {
        await _files.DeleteAsync(nombre, contenedor);
    }

    public float TamanoMb(byte[] contenido) =>
_files.GetSizeMb(contenido);
}
```

Ejemplo 2: Carga en chunks y streaming parcial

```
// C#
using Framework.FileStorage.Port;

public class CargasGrandesService
{

```

```
private readonly IFileChunkSystemPort _chunkPort;

public CargasGrandesService(IFileChunkSystemPort chunkPort) =>
    _chunkPort = chunkPort;

public async Task<string> SubirEnChunksAsync(
    string uploadId, string contenedor, string nombreArchivo,
    IEnumerable<Stream> chunks)
{
    var totalChunks = chunks.Count();

    await _chunkPort.StartUploadAsync(uploadId, contenedor,
    nombreArchivo, totalChunks);

    int i = 0;
    foreach (var chunk in chunks)
    {
        await _chunkPort.UploadChunkAsync(uploadId, contenedor,
        chunk, nombreArchivo, i++);
    }

    // Consolidación de todos los bloques en un solo blob
    return await _chunkPort.CompleteUploadAsync(uploadId,
    contenedor, nombreArchivo, totalChunks);
}

public async Task<FileResponse> ObtenerRangoAsync(
    string contenedor, string rutaArchivo, string? rangeHeader)
{
    // Ideal para endpoints de video/audio con soporte HTTP
    Range
    return await _chunkPort.GetVideoChunkAsync(contenedor,
    rutaArchivo, rangeHeader ?? string.Empty);
}
```

14.4 Casos De Uso Típicos

- Subida de archivos grandes de manera confiable y reanudable (multipart/chunks).
- Servicios de streaming de video/audio con soporte de Content-Range.
- Almacenamiento de evidencias, documentos y recursos estáticos con control de contenedor.
- Lectura eficiente a través de streams (sin cargar en memoria todo el archivo).

- Consulta de metadatos antes de iniciar descargas (por ejemplo, tamaño total para controlar límites).

14.5 Buenas Prácticas

- Validar tamaño máximo con GetSizeMb antes de guardar.
- Usar nombres únicos (por ejemplo, prefijo con GUID/fecha) para evitar colisiones.
- Implementar políticas de retención/limpieza por contenedor.
- Para streaming, propagar correctamente la cabecera Range del cliente y devolver los encabezados HTTP adecuados.
- Manejar AbortUpload para limpiar cargas incompletas/bloques huérfanos.

14.6 Ventajas De La Librería

- Desacoplamiento por contrato
 - Tu aplicación programa contra IFileSystemPort e IFileChunkSystemPort; el proveedor es intercambiable.
- Soporte para cargas grandes y streaming
 - Cargas por bloques y Content-Range facilitan UX y eficiencia en medios.
- Eficiencia y seguridad
 - Descarga por Stream evita consumir memoria innecesaria.
 - Excepciones específicas de almacenamiento permiten manejar casos esperables (duplicados, inexistentes).
- Escalabilidad
 - Azure Blob Storage (adapter incluido) permite crecer en capacidad y rendimiento sin cambios en la app.
- Configuración centralizada
 - La conexión se obtiene del sistema de configuración del framework, facilitando despliegues multi-entorno.
- Extensibilidad
 - Puedes crear nuevos adaptadores (S3, File System, GCS) implementando los puertos sin tocar la lógica de negocio.

15 FRAMEWORK.ENCRIPTOR

15.1 Introducción

Framework.Encryptor provee un puerto (interfaz) y un adaptador de cifrado simétrico para .NET 8. Su objetivo es permitir:

- Cifrar y descifrar texto de manera uniforme en toda la solución.
- Desacoplar la lógica de negocio del algoritmo/proveedor de cifrado concreto.
- Centralizar la gestión de claves y políticas de cifrado a través de configuración externa.

Modelo Port/Adapter

- Puerto: define el contrato de cifrado/descifrado.
- Adaptador: implementación concreta intercambiable sin impactar a las capas superiores.

Nota de seguridad

- Este módulo es un punto de extensión; la seguridad efectiva depende de una gestión de claves adecuada y de un algoritmo/IV/mode de operación robustos configurados correctamente.

15.2 Componentes Principales Y Tecnologías

15.2.1 Componentes

- IEncryptor (Port)
 - Encrypt(string text, string key): retorna una representación cifrada (string, típicamente Base64).
 - Decrypt(string text, string key): revierte el cifrado y devuelve el texto plano.
- Encryptor (Adapter)
 - Implementación basada en cifrado simétrico (AES).
 - Pensada como implementación por defecto y reemplazable.

15.2.2 Configuración y claves

- La clave de cifrado se obtiene típicamente desde configuración segura/secretos.
- Convención disponible: userPasswordEncryptedKey (útil como nombre de clave en el sistema de configuración).
- Recomendación: almacenar claves en un secret manager, Key Vault, o variables de entorno, nunca en código.

15.2.3 Dependencias y tecnologías

- .NET 8
- System.Security.Cryptography (AES)
- System.Text para manejo de codificación de clave y texto.

15.2.4 Buenas prácticas de configuración

- Longitud de clave acorde con el algoritmo (por ejemplo 256 bits para AES-256).
- Usar IV aleatorio por operación de cifrado y persistirlo junto al ciphertext cuando corresponda.
- Validar encoding y tamaños; manejar excepciones de cifrado.

15.3 Ejemplos De Uso Y Casos De Uso

Ejemplo de uso básico

```
// C#
using Framework.Encryptor.Port;

public class CredencialesService
{
    private readonly IEncryptor _encryptor;
    private readonly string _key;

    public CredencialesService(IEncryptor encryptor, string
keyFromConfig)
    {
        _encryptor = encryptor;
        _key = keyFromConfig;
    }

    public string GuardarSecreto(string textoPlano)
    {
        // Cifra el valor antes de persistirlo
        var cifrado = _encryptor.Encrypt(textoPlano, _key);
        // Persistir 'cifrado' en base de datos
        return cifrado;
    }

    public string LeerSecreto(string cifrado)
    {
        // Recupera y descifra el valor
        return _encryptor.Decrypt(cifrado, _key);
    }
}
```

```
}  
}
```

15.4 Casos De Uso Típicos

- Protección de datos sensibles en repositorios (p. ej., tokens, credenciales, PII).
- Cifrado de parámetros de integración antes de enviarlos por canales no confiables.
- Cifrado de archivos o fragmentos de archivo previo a su envío a un File Storage.
- Ofuscación de valores de configuración sensibles en logs o exportaciones.

15.5 Recomendaciones Operativas

- Rotación de claves: planifica cómo rotar claves de manera segura (doble descifrado/encryptado durante la transición).
- Separación por ámbito: usa diferentes claves por bounded context o por tipo de dato cuando proceda.
- Registro y auditoría: nunca loguees texto plano ni claves; registra solo metadatos (id, timestamp).

15.6 VENTAJAS DE LA LIBRERÍA

- Desacoplamiento
 - La aplicación depende del puerto (IEncryptor); puedes cambiar la implementación sin alterar la lógica.
- Sencillez de integración
 - API mínima y clara (Encrypt/Decrypt) para cubrir los casos comunes de cifrado simétrico.
- Extensibilidad
 - Permite introducir nuevas implementaciones (por ejemplo, AES-GCM con IV aleatorio, HSM, o servicios KMS) cumpliendo el contrato.
- Consistencia
 - Evita soluciones ad-hoc de cifrado repartidas por el código; centraliza y uniforma el enfoque.
- Preparado para entornos regulados
 - Facilita políticas de seguridad (gestión de claves, rotación, segregación de funciones) al mantener un punto de extensión único.

16 FRAMEWORK.VALIDATIONS

16.1 Introducción

Framework.Validations estandariza la validación de datos en la capa de aplicación y dominio mediante una fachada simple sobre FluentValidation. Provee:

- Un contrato único (IValidator) para ejecutar validaciones asíncronas.
- Integración con el contenedor de dependencias del framework.
- Excepciones de validación uniformes para manejo de errores y respuesta consistente.

16.2 OBJETIVO

Centralizar la validación, reducir duplicidad y asegurar mensajes de error homogéneos en toda la solución.

16.3 Componentes Principales Y Tecnologías

16.3.1 Componentes

- IValidator
 - Método ValidAsync<TObject, TValidator>(obj): ejecuta un validador específico para el tipo del objeto.
- Validator
 - Implementación que resuelve el TValidator desde el contenedor (DI), valida y lanza una ValidationException en caso de errores.
- ValidationException
 - Excepción específica con lista de errores (código y mensaje), compatible con el modelo de errores transaccionales del framework (TransactionException).

16.3.2 Tecnologías y dependencias

.NET 8

- FluentValidation
- Inyección de dependencias (IScopeProvider / ServiceCollection)
- Modelo de errores transaccionales del framework (Framework.Transaction.Exceptions)

16.3.3 Convenciones

- Un validador por DTO/comando/consulta con reglas declarativas de FluentValidation.
- Errores con código y mensaje listos para serializar en respuestas de API.

16.4 EJEMPLOS DE USO Y CASOS DE USO

Ejemplo 1: Validar un comando antes de ejecutar lógica

```
// C#
using FluentValidation;
using Framework.Validations;

public class CrearUsuarioCommand
{
    public string Email { get; set; } = default!;
    public string Password { get; set; } = default!;
}

public class CrearUsuarioValidator :
    AbstractValidator<CrearUsuarioCommand>
{
    public CrearUsuarioValidator()
    {
        RuleFor(x => x.Email).NotEmpty().EmailAddress();
        RuleFor(x => x.Password).NotEmpty().MinimumLength(8);
    }
}

public class CrearUsuarioHandler
{
    private readonly IValidator _validator;
    public CrearUsuarioHandler(IValidator validator) => _validator
    = validator;

    public async Task HandleAsync(CrearUsuarioCommand cmd)
    {
        await _validator.ValidateAsync<CrearUsuarioCommand,
        CrearUsuarioValidator>(cmd);
        // Si llega aquí, las reglas se cumplieron. Ejecutar la
        lógica de negocio.
    }
}
```

Ejemplo 2: Integración con pipeline CQRS

```
// C#
using FluentValidation;
using Framework.Cqrs.Services;
using Framework.Validations;

public class ActualizarPerfilCommand
{
    public Guid UsuarioId { get; set; }
    public string Nombre { get; set; } = default!;
}

public class ActualizarPerfilValidator :
    AbstractValidator<ActualizarPerfilCommand>
{
    public ActualizarPerfilValidator()
    {
        RuleFor(x => x.UsuarioId).NotEmpty();
        RuleFor(x => x.Nombre).NotEmpty().MaximumLength(100);
    }
}

public class ActualizarPerfilHandler :
    ICommandHandler<ActualizarPerfilCommand>
{
    private readonly IValidator _validator;
    public ActualizarPerfilHandler(IValidator validator) =>
        _validator = validator;

    public async Task HandleAsync(ActualizarPerfilCommand command)
    {
        await _validator.ValidAsync<ActualizarPerfilCommand,
            ActualizarPerfilValidator>(command);
        // Persistencia y lógica...
    }
}
```

16.5 Casos De Uso Típicos

- Validación de comandos y queries en CQRS antes de tocar el dominio.
- Validación de DTOs de entrada de APIs.
- Composición de validaciones complejas con dependencias resolubles por DI (por ejemplo, verificar unicidad en base de datos).

16.6 Buenas Prácticas

- Mantén las reglas en validadores dedicados (Single Responsibility).
- Usa códigos de error significativos para clientes (catálogo de errores).
- Evita lógica de negocio en validadores; limita a reglas sintácticas y semánticas de entrada.
- Centraliza el manejo de `ValidationException` para producir respuestas HTTP coherentes (422/400).

16.7 Ventajas De La Librería

- Consistencia en errores
 - Una sola forma de validar y reportar fallos, con estructura uniforme de mensajes y códigos.
- Desacoplamiento y testabilidad
 - Validadores independientes y testeables; los handlers solo orquestan la llamada a `IValidator`.
- Integración fluida
 - Se conecta con DI, CQRS y el modelo de errores transaccionales del framework.
- Productividad
 - Aprovecha `FluentValidation` para expresar reglas de manera declarativa y legible.
- Escalabilidad de reglas
 - Puedes componer reglas, reutilizar validadores y agregar dependencias sin acoplarte a infraestructura específica.

17 FRAMEWORK.CACHE

17.1 INTRODUCCIÓN

Framework.Cache provee contratos para cachear datos y valores en aplicaciones .NET 8 de forma agnóstica al proveedor. Su objetivo es:

- Unificar cómo se leen/escriben entradas de caché tipadas.
- Separar modelos cacheables (`ICacheModel`) de valores arbitrarios (`ICacheValues`).
- Permitir selección de base/espacio lógico de caché mediante atributos.

Se integra con adapters específicos (por ejemplo, Redis) que implementan estos contratos, sin acoplar la lógica de aplicación a un proveedor concreto.

17.2 Componentes Principales Y Tecnologías

17.2.1 Componentes

- ICacheModel
 - Contrato para modelos cacheables; exige una Key única por entrada.
- ICacheServiceServicio para trabajar con modelos cacheados (colecciones y entradas individuales). Operaciones: GetAsync(key): obtiene un modelo por clave. GetAllAsync(): lista todas las entradas del tipo. GetAllByQuery(query): recupera por patrón/consulta (según soporte del adapter). SaveCacheModelAsync(model/s, duration): guarda uno o varios modelos con TTL.
- ICacheValues
 - Cache de valores arbitrarios (clave/valor genérico).
 - Operaciones:
 - GetValueAsync(key)
 - SetValueAsync(key, value, timeSpan)
- DbAttribute
 - Atributo para anotar una clase cacheable con un índice/base lógico (db) cuando el backend lo soporta (por ejemplo, bases de datos lógicas en Redis).

17.2.2 Tecnologías y dependencias

.NET 8: Los adapters típicos usan Redis como backend, pero el diseño es agnóstico.

17.2.3 Convenciones

- La propiedad Key debería ser estable y derivada del identificador funcional del dato.
- El tiempo de vida (TTL) debe reflejar la frescura requerida por el negocio.

17.3 Ejemplos De Uso Y Casos De Uso

Ejemplo 1: Cachear modelos tipados

```
// C#  
using Framework.Cache;  
using Framework.Cache.Model;  
  
[Db(1)]
```

```
public class ProductoCache : ICacheModel
{
    public string Key { get; set; } = default!;
    public string Nombre { get; set; } = default!;
    public decimal Precio { get; set; }
}

public class CatalogoService
{
    private readonly ICacheService<ProductoCache> _cache;
    public CatalogoService(ICacheService<ProductoCache> cache) =>
        _cache = cache;

    public async Task GuardarProductoAsync(ProductoCache model)
        => await _cache.SaveCacheModelAsync(model,
        TimeSpan.FromMinutes(30));

    public async Task<ProductoCache?> ObtenerProductoAsync(string
    key)
        => await _cache.GetAsync(key);

    public async Task<IEnumerable<ProductoCache>>
    ListarProductosAsync()
        => await _cache.GetAllAsync();
}
```

Ejemplo 2: Cache clave/valor genérico

```
// C#
using Framework.Cache;

public class FeatureFlagsService
{
    private readonly ICacheValues _values;
    public FeatureFlagsService(ICacheValues values) => _values =
    values;

    public async Task<bool> EstaHabilitadoAsync(string flag)
        => (await _values.GetValueAsync<bool?>($"feature:{flag}"))
        ?? false;

    public Task HabilitarAsync(string flag)
        => _values.SetValueAsync($"feature:{flag}", true,
        TimeSpan.FromHours(1));
}
```

17.4 Casos De Uso Típicos

- Cache de resultados de queries costosas o de proyecciones leídas frecuentemente.
- Feature flags, tokens temporales, conteos o valores simples con expiración.
- Lookup tables o catálogos que cambian poco.
- Rate limiting o locks ligeros basados en claves.

17.5 Buenas Prácticas

- Diseña claves únicas y consistentes (prefijos por bounded context).
- Define TTL apropiados y política de invalidación explícita.
- Evita cachear información altamente volátil sin un mecanismo de invalidación por evento.
- En escenarios multientorno, separa espacios lógicos (DbAttribute o prefijos) para evitar colisiones.

17.6 Ventajas De La Librería

- Desacoplamiento del backend de caché
 - Tu código depende de interfaces; puedes cambiar Redis por otro backend con un nuevo adapter.
- Tipado y claridad
 - Diferencia explícita entre modelos cacheados y valores sueltos; más legibilidad y control.
- Escalabilidad y rendimiento
 - Reduce latencias y carga en la capa de datos al reutilizar resultados.
- Flexibilidad operativa
 - Control de TTL y partición lógica (db/prefijos) por tipo de dato o contexto.
- Integración natural
 - Fácil de combinar con CQRS y EDA para invalidar/actualizar cache ante eventos de dominio/integración.

18 FRAMEWORK.REPOSITORY

18.1 Introducción

Framework.Repository define contratos genéricos de persistencia para .NET 8, alineados con arquitectura limpia y DDD. Su objetivo es:

- Desacoplar la capa de aplicación y dominio de la tecnología de almacenamiento (EF Core, MongoDB, etc.).
- Establecer un conjunto homogéneo de operaciones CRUD y de consulta por expresiones.
- Integrarse con el modelo transaccional del framework para operaciones atómicas y orquestación de UoW.

18.2 Componentes Principales Y Tecnologías

18.2.1 Componentes

- IEntity
 - Contrato que deben implementar las entidades persistentes; requiere Id tipado y comparable.
- IRepository<TEntity, TId>
 - Contrato genérico para acceso a datos con soporte para:
 - InsertAsync, InsertBulkAsync, InsertAndUpdateAsync.
 - UpdateAsync, DeleteByIdAsync.
 - GetAllAsync, GetByIdAsync, GetFirstByFilterAsync.
 - GetByFilterAsync, GetByFilterPageAsync con expresiones LINQ (Expression<Func<...>>).
 - Hereda ITransactionChild para integrarse con transacciones distribuidas/ambientales del framework.
 - Extiende IDisposable para liberar recursos del provider subyacente.
- Enums.DbType
 - Enum de conveniencia para seleccionar proveedores en adapters (Ef, MongoDB).
- RepositoryException
 - Excepción específica para errores de repositorio.

18.2.2 Tecnologías y dependencias

- .NET 8
- Expresiones LINQ para filtros y paginación.
- Integración con Framework.Transaction para gestionar información contextual y alcance transaccional.

18.2.3 Convenciones

- TEntity debe implementar IEntity.
- TId debe ser comparable (IComparable) para consistencia en operaciones de identidad.

18.3 Ejemplos De Uso Y Casos De Uso

Ejemplo 1: Uso básico en un servicio de aplicación

```
// C#
using Framework.Repository;

public class Producto : IEntity<Guid>
{
    public Guid Id { get; set; }
    public string Nombre { get; set; } = default!;
    public decimal Precio { get; set; }
}

public class CatalogoService : IDisposable
{
    private readonly IRepository<Producto, Guid> _repo;
    public CatalogoService(IRepository<Producto, Guid> repo) =>
        _repo = repo;

    public async Task<Guid> CrearAsync(string nombre, decimal
precio)
    {
        var entity = new Producto { Id = Guid.NewGuid(), Nombre =
nombre, Precio = precio };
        await _repo.InsertAsync(entity);
        return entity.Id;
    }

    public Task<IEnumerable<Producto>> BuscarPorNombreAsync(string
texto)
        => _repo.GetByFilterAsync(x => x.Nombre.Contains(texto));
}
```

```
public Task<Producto?> ObtenerAsync(Guid id) =>
_repo.GetByIdAsync(id);

public async Task ActualizarPrecioAsync(Guid id, decimal
nuevoPrecio)
{
    var entity = await _repo.GetByIdAsync(id) ?? throw new
InvalidOperationException("No encontrado");
    entity.Precio = nuevoPrecio;
    await _repo.UpdateAsync(entity);
}

public Task EliminarAsync(Guid id) =>
_repo.DeleteByIdAsync(id);

public void Dispose() => _repo.Dispose();
}
```

Ejemplo 2: Paginación por filtro

```
// C#
public Task<IEnumerable<Producto>> BuscarPaginadoAsync(string
texto, int page, int pageSize)
=> _repo.GetByFilterPageAsync(x => x.Nombre.Contains(texto),
page, pageSize);
```

18.4 CASOS DE USO TÍPICOS

- CRUD de entidades del dominio o modelos de lectura.
- Consultas con filtros complejos mediante expresiones.
- Operaciones en lote (InsertBulkAsync) para importaciones masivas.
- Integración con UoW/Transacciones mediante ITransactionChild.

18.5 BUENAS PRÁCTICAS

- Mantén las consultas complejas encapsuladas en repositorios específicos o query objects.
- Usa InsertBulkAsync para operaciones de alto volumen.
- Maneja RepositoryException para mapear a errores de aplicación o API.
- Aísla lógica de dominio de la persistencia; el repositorio solo persiste y recupera.

18.6 Ventajas De La Librería

- Desacoplamiento de la infraestructura

- La aplicación depende del contrato; EF Core, MongoDB u otros se conectan por adapters.
- Flexibilidad de consultas
 - Filtros mediante expresiones LINQ permiten composibilidad y reutilización.
- Integración transaccional
 - Herencia de `ITransactionChild` para participar en transacciones coordinadas.
- Consistencia API
 - Un conjunto uniforme de operaciones agiliza el desarrollo y reduce curva de aprendizaje.
- Testabilidad
 - Fácil de mockear para pruebas de aplicación y dominio sin tocar la base de datos.

19 FRAMEWORK.MESSAGEBUS

19.1 introducción

Framework.MessageBus define los contratos y utilidades para la publicación y suscripción de mensajes en arquitecturas orientadas a eventos y mensajería en .NET 8. Su objetivo es:

- Desacoplar la aplicación del broker específico (RabbitMQ, etc.) mediante interfaces.
- Estandarizar el contrato de mensajes y handlers.
- Proveer un mediador de suscriptores que resuelve handlers, gestiona el contexto transaccional y ejecuta la lógica con seguridad.

Este módulo es la base sobre la cual se integran adapters concretos de transporte (por ejemplo, RabbitMQ) y se conecta con Framework.Eda para publicar eventos de integración utilizando Outbox.

19.2 Componentes Principales Y Tecnologías

19.2.1 Componentes

- IMessage
 - Contrato mínimo de mensaje: Id, PublishDate e Information (metadatos transaccionales y de correlación).

- IMessageHandlerContrato que implementan los consumidores de mensajes. Expone HandleAsync(TMessage).
- IPublisher
 - Abstracción de alto nivel para publicar mensajes tipados. Serializa y envía según atributos del mensaje.
- IPublishers
 - Abstracción de bajo nivel del publisher hacia el broker. Publica por topic, routingKey y payload (JSON) y devuelve éxito/fracaso.
- Publisher
 - Implementación de IPublisher:
 - Obtiene metadatos de publicación por atributo PublisherAttribute en la clase del mensaje.
 - Serializa con convenciones del módulo y publica a través de IPublishers.
- ISubscribers
 - Contrato para registrar suscripciones y comenzar a escuchar (Listen) en el transporte subyacente.
- ISubscriberMediator
 - Orquestador que, al recibir un mensaje crudo (nombre del mensaje, payload, clave), resuelve el handler correspondiente y lo invoca dentro de un alcance transaccional.
- SubscriberRegister
 - Registro singleton que mapea nombres de mensajes a tipos de handlers. Es utilizado por el mediador para la resolución.
- Atributos
 - PublisherAttribute(messageName): declara el nombre del mensaje para publicación.
 - SubscribeAttribute(serviceEmitter, messageName, forceSubscribe opcional): declara a qué mensajes se suscribe un handler y si aplica restricciones de auto-resolución.
 - SubscribeHandleAttribute(key?): permite distinguir instancias de handlers por clave, útil para múltiples suscriptores del mismo mensaje.

19.2.2 Tecnologías y dependencias

.NET 8

- System.Text.Json para serialización
 - Integración con:
 - Framework.Scaffolding (IProvider) para AppName y creación de scopes.
 - Framework.Transaction (ITransaction, Information) para contexto transaccional, commit/rollback alrededor del handler.
 - Framework.Logger para registrar errores en el procesamiento.
- Adapters externos conectan IPublishers e ISubscribers con el broker elegido.

19.3 Ejemplos De Uso Y Casos De Uso

Ejemplo 1: Publicar un mensaje tipado

```
// C#
using Framework.MessageBus;
using Framework.MessageBus.Attributes;
using Framework.MessageBus.Models;

[Publisher("Usuarios.Creado")]
public class UsuarioCreadoMessage : IMessage
{
    public Guid Id { get; set; } = Guid.NewGuid();
    public DateTime PublishDate { get; set; } = DateTime.UtcNow;
    public Information? Information { get; set; }
    public Guid UsuarioId { get; set; }
    public string Email { get; set; } = default!;
}

public class NotificacionesService
{
    private readonly IPublisher _publisher;
    public NotificacionesService(IPublisher publisher) =>
        _publisher = publisher;

    public Task EnviarAvisoAsync(Guid usuarioId, string email,
        Information info)
        => _publisher.PublishAsync(new UsuarioCreadoMessage {
            UsuarioId = usuarioId, Email = email, Information = info });
}
```

Ejemplo 2: Suscripción y mediación de mensajes

```
// C#
using Framework.MessageBus;
using Framework.MessageBus.Attributes;
using Framework.MessageBus.Models;

[Subscribe("App.Productos", "Usuarios.Creado")]
public class EnviarDescuentoHandler :
    IMessageHandler<UsuarioCreadoMessage>
{
    public Task HandleAsync(UsuarioCreadoMessage message)
    {
        // Lógica de negocio al recibir el mensaje
        return Task.CompletedTask;
    }
}

// En el adapter del broker se registra la suscripción y se delega
a ISubscriberMediator
public class WorkerSuscripciones
{
    private readonly ISubscribers _subs;
    public WorkerSuscripciones(ISubscribers subs) => _subs = subs;

    public void Iniciar()
    {
        _subs.Subscribe(topic: "App.Productos", messageName:
            "Usuarios.Creado", key: "descuentos");
        _subs.Listen();
    }
}
```

19.4 FLUJO DE SUSCRIPCIÓN CON MEDIADOR

- El adapter recibe un mensaje del broker y obtiene messageName y payload.
- Llama a ISubscriberMediator.MediateAsync(messageName, payload, key).
- El mediador:
 - Crea un scope de DI y resuelve ITransaction y el handler asociado al messageName.
 - Deserializa el payload al tipo de mensaje esperado.
 - Propaga Information a la transacción si está presente.
 - Invoca HandleAsync y, si todo va bien, Commit; en errores, Rollback y log.

19.5 Casos De Uso Típicos

- Integración entre servicios mediante eventos de integración.
- Orquestación de workflows asíncronos con entrega al menos una vez.
- Fan-out de mensajes a múltiples consumidores con claves/instancias diferenciadas.
- Pipelines de procesamiento con lógica transaccional y reintentos controlados por el broker.

19.6 Buenas Prácticas

- Mensajes inmutables, explícitos y con nombres canónicos.
- Idempotencia en handlers: tolerar reentregas.
- Observabilidad: log y métricas por messageName, tiempos, intentos y resultados.
- Backpressure: configura QoS y concurrencia en el adapter del broker.

19.7 Ventajas De La Librería

- Desacoplamiento del transporte
 - Publicadores y suscriptores programan contra interfaces, no contra APIs de broker.
- Mediación transaccional
 - El ISubscriberMediator rodea el handler con una transacción y manejo de errores coherente.
- Convenciones fuertes
 - Atributos definen claramente nombres de mensajes y suscripciones, facilitando descubrimiento y registro.
- Extensibilidad
 - Permite agregar adapters para distintos brokers, customizar serialización y políticas de consumo.
- Observabilidad y gobernanza
 - Al propagar Information y usar un registro central de handlers, facilita auditoría y troubleshooting.

20 FRAMEWORK.SCAFFOLDING

20.1 INTRODUCCIÓN

Framework.Scaffolding ofrece un mecanismo de ensamblaje y auto-registro de componentes en .NET 8, con enfoque en:

- Descubrimiento de módulos y secciones a partir de ensamblados.
- Registro de servicios en el contenedor de DI con ciclos de vida controlados.
- Creación de scopes en tiempo de ejecución y resolución segura de dependencias.
- Hooks para ejecutar lógica de “post-ensamblaje”.

20.2 Objetivo

Estandarizar cómo los módulos del framework (y externos) se registran y se conectan entre sí, minimizando configuración manual.

20.3 Qué Resuelve

- Autoescaneo de DLLs “Framework.*” y opcionalmente proyectos externos.
- Registro centralizado vía IService e integración con configuración.
- Resolución tipada (y por clave) tanto en raíz como en scopes.

20.4 COMPONENTES PRINCIPALES Y TECNOLOGÍAS

20.4.1 Componentes

- Assembler
 - Escanea directorios de binarios, carga tipos públicos y ejecuta secciones encontradas.
 - Work(config): invoca ISectionService.JoinSection para permitir registro de servicios por módulo.
 - Joined(provider): invoca ISectionJoined.Joined tras el registro para ejecutar inicialización tardía.
 - Permite incluir DLLs adicionales mediante parámetros dllsSearch.
- IService
 - API de registro en el contenedor:
 - Register(lifeCycle, key?)
 - Register<TService, TImp>(lifeCycle, key?)
 - Register(lifeCycle, Type service, Type imp, key?)

- RegisterInstance(instance, key?)
 - Soporta ciclos de vida: Singleton, Scope, Transient.
- IProvider
 - Proveedor raíz de servicios:
 - AppName
 - Resolve/ResolveRequired por tipo y clave.
 - CreateScope(): crea un IServiceProvider.
- IServiceProvider
 - Scope de DI con capacidades:
 - Resolve/ResolveRequired tipado y por tipo.
 - ResolverAll() / ResolverAll(Type): múltiples implementaciones.
 - AppName (propagado).
- ISectionService
 - Contrato que implementan “secciones” de registro por módulo.
 - JoinSection(IService, IEnumerable, IScaffConfiguration): registra servicios según convenciones y configuración.
- ISectionJoined
 - Hook posterior al ensamblaje para ejecutar lógica que necesita el contenedor listo (mapeos, warmups, inicializaciones).
- Lifecycle
 - Enum: Singleton, Scope, Transient.

20.4.2 Tecnologías y dependencias

.NET 8

- Integración con Framework.Configuration (IScaffConfiguration).
- Integración natural con módulos del framework que exponen secciones de registro.

20.5 Ejemplos De Uso Y Casos De Uso

Ejemplo: Ensamblado y registro de módulos

```
// C#  
using Framework.Scaffolding;  
using Framework.Configuration;
```

```
public class Arranque
{
    private readonly IService _service;
    private readonly IScaffConfiguration _config;

    public Arranque(IService service, IScaffConfiguration config)
    {
        _service = service;
        _config = config;
    }

    public void Boot()
    {
        // Escanear Framework.* y DLLs externas opcionales (por
        patrón de nombre)
        var assembler = new Assembler(_service, "MyCompany.",
        "MyApp.");
        assembler.Work(_config);

        // Crear provider raíz a partir de IService (implementación
        específica)
        var provider = /* construir IProvider a partir del
        contenedor */;
        assembler.Joined(provider);
    }
}
```

Ejemplo: Definir una sección de registro

```
// C#
using Framework.Scaffolding;
using Framework.Scaffolding.Enums;
using Framework.Configuration;

public class SeccionUsuarios : ISectionService
{
    public void JoinSection(IService service, IEnumerable<Type>
    types, IScaffConfiguration config)
    {
        // Registro de servicios del módulo Usuarios
        service.Register<IUsuarioRepository,
        UsuarioRepository>(LifeCycle.Scope);
        service.Register<IUsuarioService,
        UsuarioService>(LifeCycle.Transient);
        // Registro por clave (si hay múltiples implementaciones)
        service.Register(typeof(INormalizadorNombre),
```

```
typeof(NormalizadorEs), Lifecycle.Singleton, key: "es");
    }
}

public class UsuariosJoined : ISectionJoined
{
    public void Joined(IProvider provider)
    {
        // Lógica post-ensamblaje (e.g., precalcular caches,
        registrar mapeos, métricas)
        var srv = provider.ResolveRequired<IUsuarioService>();
        // srv.Warmup();
    }
}
```

20.6 Casos De Uso Típicos

- Auto-registro de módulos de dominio, infraestructura y APIs sin wiring manual.
- Multi-implementaciones por clave para estrategias, validadores, normalizadores, etc.
- Inicialización tardía (Joined) para construir índices, precargar catálogos o enlazar suscripciones.

20.7 Buenas Prácticas

- Un módulo = una ISectionService con registro explícito y documentado.
- Usar Lifecycle adecuado: Singleton para stateless/pesado, Scope para servicios que participan en transacciones, Transient para componentes livianos.
- Preferir interfaces y registros por contrato; evita acoplar a clases concretas.
- Mantener Joined libre de efectos costosos en caliente; si necesitas cálculos pesados, hazlos en background.

20.8 Ventajas De La Librería

- Descubrimiento automático
 - Carga y registra módulos sin configuración manual extensa; escala bien en soluciones grandes.
- Consistencia y orden
 - Convenciones de registro y hooks homogéneos entre módulos del framework y módulos propios.
- Flexibilidad

- Soporte de múltiples implementaciones por tipo (con clave) y de distintos ciclos de vida.
- Integración transversal
 - Funciona con configuración, transacciones, logging y el resto de módulos del framework.
- Productividad
 - Menos boilerplate de arranque, más foco en la lógica de negocio.

21 FRAMEWORK.TRANSACTION

21.1 Introducción

Framework.Transaction proporciona un contexto transaccional transversal para la aplicación. Permite:

- Propagar metadatos de la transacción (Information) como usuario, token, zona horaria, idioma, etc.
- Coordinar y ordenar múltiples participantes transaccionales (ITransactionChild) con commit/rollback.
- Encapsular recursos desechables y asegurar su liberación al finalizar el alcance.
- Integrarse con DI (scopes) para ejecutar operaciones dentro de un contexto coherente.

Este módulo es clave para mantener consistencia y trazabilidad en operaciones complejas, y se usa en componentes como MessageBus y Eda.

21.2 Componentes Principales Y Tecnologías

21.2.1 Componentes

- Information
 - Contiene TransactionId, Authorization/Token, TimeZone, OS, idioma (Lang), moneda, device, browser, IP, NotificationToken y el User asociado.
- User
 - Representa información del usuario (Id, nombres y un diccionario Data con claims/atributos).
- ITransaction
 - API del contexto transaccional:

- Id, Information, TimeZoneInfo, ScopeProvider.
 - AddTransactionInformation(...): llena Information, obtiene User opcionalmente con IGetUserService y construye el TimeZoneInfo.
 - CreateTimeZone(): genera una zona horaria personalizada a partir de Information.TimeZone.
 - AddTransactionChild(ITransactionChild): registra participantes y recursos desechables.
 - GetTransactionalChilds(object): descubre y registra automáticamente participantes presentes en el grafo de dependencias de un objeto.
 - CommitAsync / RollbackAsync: orquesta a los hijos por orden.
 - AddDispose: registra recursos a limpiar.
- ITransactionChild
 - Participante de la transacción, con Order, CommitAsync y RollbackAsync.
 - OrderAttribute
 - Permite priorizar el orden de ejecución de los hijos durante Commit/Rollback.
 - IGetUserService
 - Contrato para obtener datos del usuario actual a partir de Information (opcional).
 - IValidSecurity
 - Contrato para validar roles/autorización (usable por pipelines o atributos).

21.2.2 Tecnologías y dependencias

.NET 8

- Integración con Framework.Scaffolding (IProvider/IScopeProvider para scopes y resolución).
- Extensiones utilitarias para iteraciones asíncronas.

21.3 Ejemplos De Uso Y Casos De Uso

Ejemplo 1: Población de información y uso básico

```
// C#  
using Framework.Transaction;  
  
public class TransaccionService
```

```
{
    private readonly ITransaction _tx;
    public TransaccionService(ITransaction tx) => _tx = tx;

    public async Task EjecutarAsync()
    {
        _tx.Id = Guid.NewGuid();
        await _tx.AddTransactionInformation(
            browser: "Chrome",
            moneyCode: "USD",
            language: "es",
            device: "Web",
            ip: "10.0.0.1",
            notificationToken: "n/a",
            os: "Windows",
            timeZone: "UTC-05:00",
            token: "jwt-token",
            getUser: null
        );

        // ... trabajo de aplicación ...

        await _tx.CommitAsync();
    }
}
```

Ejemplo 2: Registrar participantes transaccionales

```
// C#
using Framework.Transaction;

[Order(10)]
public class RepoUnit : ITransactionChild
{
    public int? Order { get; set; }
    public Task CommitAsync() => Task.CompletedTask; // p. ej.,
    SaveChangesAsync()
    public Task RollbackAsync() => Task.CompletedTask; // p. ej.,
    revertir/limpiar
}

public class CasoDeUso
{
    private readonly ITransaction _tx;
    public CasoDeUso(ITransaction tx) => _tx = tx;
```

```
public async Task RunAsync()
{
    var repo = new RepoUnit();
    _tx.AddTransactionChild(repo);

    // O bien descubrir automáticamente en un grafo de objetos
    _tx.GetTransactionalChilds(this);

    try
    {
        // ... ejecutar operaciones ...
        await _tx.CommitAsync();
    }
    catch
    {
        await _tx.RollbackAsync();
        throw;
    }
    finally
    {
        _tx.Dispose(); // limpia disposables registrados
    }
}
```

21.4 CASO DE USO: INTEGRACIÓN CON MESSAGEBUS/EDA

- SubscriberMediator crea un scope, inyecta ITransaction y lo usa para:
 - Propagar Information del mensaje.
 - Descubrir participantes transaccionales del handler.
 - Ejecutar el handler y, al terminar, Commit o Rollback y logging.

21.5 Ventajas De La Librería

- Consistencia transaccional
 - Coordina múltiples recursos (repositorios, buses, outbox) bajo un mismo contexto y orden definido.
- Trazabilidad y observabilidad
 - Information trae metadatos clave para correlación, auditoría y localización (zona horaria).
- Productividad y seguridad
 - Descubrimiento automático de ITransactionChild reduce boilerplate.

- Orden explícito con atributos evita dependencias implícitas y errores.
- Integración transversal
 - Funciona junto a CQRS, EDA, MessageBus y Repositorios, unificando commit/rollback y metadatos.

22 FRAMEWORK.LOGGER

22.1 Introducción

Framework.Logger define un contrato mínimo de logging para desacoplar la aplicación de cualquier proveedor de logs (por ejemplo, ILogger de .NET, Serilog, NLog). El objetivo es ofrecer:

- Una interfaz simple con niveles estándar (Trace, Debug, Info, Warn, Error, Critical).
- Métodos que acepten mensajes con formato y parámetros.
- Sobrecargas para registrar excepciones junto al mensaje.

Esto permite cambiar la implementación concreta sin afectar el resto del código y unificar la forma de loggear en todos los módulos del framework.

22.2 COMPONENTES PRINCIPALES Y TECNOLOGÍAS

22.2.1 Componente

- ILogger
 - Info, Trace, Warn, Debug: para información, trazas detalladas, advertencias y depuración.
 - Error: dos variantes, con y sin Exception.
 - Critical: dos variantes, con y sin Exception.
 - Firma homogénea: (string? message, params object?[] args) para dar soporte a formateo perezoso e interpolación.

22.2.2 Tecnologías y dependencias

.NET 8

- Las implementaciones concretas suelen adaptarse a Microsoft.Extensions.Logging, Serilog, NLog, etc.

22.2.3 Convenciones

- Mensajes estructurados y uso de parámetros para propiedades clave (Ids, nombres, estados).

- Evitar loggear datos sensibles; aplicar políticas de ofuscación cuando necesario.

22.3 EJEMPLOS DE USO Y CASOS DE USO

Ejemplo: Uso desde un servicio de aplicación

```
// C#
using Framework.Logger;

public class PedidosService
{
    private readonly ILogger _log;
    public PedidosService(ILogger log) => _log = log;

    public void Procesar(Guid pedidoId)
    {
        _log.Info("Iniciando procesamiento de pedido {PedidoId}",
pedidoId);
        try
        {
            // ... lógica ...
            _log.Debug("Detalle de pasos intermedios para
{PedidoId}", pedidoId);
            _log.Info("Pedido {PedidoId} procesado correctamente",
pedidoId);
        }
        catch (Exception ex)
        {
            _log.Error(ex, "Error procesando pedido {PedidoId}",
pedidoId);
            throw;
        }
    }
}
```

22.4 Casos De Uso Típicos

- Trazabilidad de comandos/queries y handlers.
- Registro de eventos de integración publicados/consumidos.
- Métricas basadas en logs y auditoría operativa.
- Diagnóstico de errores con stacktrace y contexto.

22.5 Buenas Prácticas

- Incluir identificadores de correlación (RequestId, TransactionId) en los mensajes.
- Definir niveles adecuados para evitar ruido o silencios.

- Estructurar los logs (parámetros) para facilitar consultas en observabilidad.
- No registrar datos personales sensibles sin enmascaramiento.

22.6 Ventajas De La Librería

- Desacoplamiento del proveedor
 - Permite cambiar/eliminar dependencias concretas de logging sin refactor masivo.
- Consistencia transversal
 - Un único contrato compartido por todos los módulos del framework.
- Testabilidad
 - Fácil de mockear/verificar en pruebas unitarias y de integración.
- Preparado para observabilidad
 - Uso de parámetros favorece logs estructurados y analíticas posteriores.

23 FRAMEWORK.CONFIGURATION

23.1 INTRODUCCIÓN

Framework.Configuration define un contrato mínimo para acceder a valores y secciones de configuración en .NET 8. Permite desacoplar la lógica de aplicación de un proveedor concreto (por ejemplo, IConfiguration de Microsoft) y mantener una API uniforme para leer claves y vincular secciones tipadas.

23.2 Objetivo

Simplificar y estandarizar el acceso a configuración en los módulos del framework y en proyectos de aplicación.

23.3 COMPONENTES PRINCIPALES Y TECNOLOGÍAS

23.3.1 Componente

- IScaffConfiguration
 - GetValue(key): obtiene un valor simple convertido al tipo T.
 - GetSection(key): enlaza una sección de configuración a un objeto/record T (options tipadas).

23.3.2 Tecnologías y dependencias

.NET 8

- Los adapters suelen implementar esta interfaz sobre Microsoft.Extensions.Configuration.
- Compatible con variables de entorno, archivos JSON/YAML, secretos, etc., según el adapter.

23.3.3 Convenciones

- Claves de configuración con namespaces lógicos por módulo (p. ej., filestorage.connectionstring, messagebus.rabbitmq.host).
- Secciones para opciones complejas (p. ej., Outbox, RetryPolicy, Storage, etc.).

23.4 Ejemplos De Uso Y Casos De Uso

Ejemplo 1: Obtener valores simples

```
// C#
using Framework.Configuration;

public class ServiceConConfiguracion
{
    private readonly string _connString;

    public ServiceConConfiguracion(IScaffConfiguration config)
    {
        _connString =
            config.GetValue<string>("filestorage.connectionstring") ?? "";
    }
}
```

Ejemplo 2: Secciones tipadas

```
// C#
using Framework.Configuration;

public class CacheOptions
{
    public int DefaultTtlMinutes { get; set; }
    public bool Enabled { get; set; }
}

public class CacheService
{
    public CacheService(IScaffConfiguration config)
    {
        var options = config.GetSection<CacheOptions>("cache");
        // options?.DefaultTtlMinutes, options?.Enabled, etc.
    }
}
```

```
}  
}
```

23.5 Casos De Uso Típicos

- Proveer cadenas de conexión a adaptadores (FileStorage, Repository, MessageBus).
- Configurar parámetros de operación para jobs (frecuencia, lotes, límites).
- Activar/desactivar funcionalidades por entorno mediante flags.

23.6 Buenas Prácticas

- Centraliza claves y nombres de sección en constantes por módulo.
- Define modelos de opciones con valores por defecto razonables.
- Evita valores mágicos; preferir configuración sobre hardcode.
- Mantén convenciones consistentes entre entornos (Dev/QA/Prod).

23.7 Ventajas De La Librería

- Desacoplamiento del proveedor de configuración
 - Cambia la fuente de configuración sin alterar el resto del código.
- Consistencia
 - Acceso uniforme para todos los módulos del framework.
- Testabilidad
 - Fácil de mockear para pruebas unitarias al simular configuraciones.
- Seguridad y gobernanza
 - Facilita ubicar y documentar claves críticas; apoya el uso de secretos externos.

24 FRAMEWORK.UTILS

24.1 Introducción

Framework.Utils proporciona utilidades generales y extensiones de uso común para proyectos .NET 8 del framework. Su objetivo es reducir boilerplate y aumentar la legibilidad en operaciones frecuentes sobre colecciones y expresiones.

24.2 Componentes Principales Y Tecnologías

- Extensiones de colección

- `ForEach(IEnumerable, Action)` Itera una secuencia y ejecuta una acción por elemento.
 - `ForEachAsync(IEnumerable, Func)` Itera una secuencia ejecutando una operación asíncrona por elemento en orden secuencial.
- Extensiones de expresiones
 - `GetPropertyNames(Expression >)` Devuelve el nombre de la propiedad referenciada por una expresión lambda. Soporta casos en los que el cuerpo es un `UnaryExpression` (por ejemplo, conversión implícita a `object`).

24.2.1 Tecnologías y dependencias

.NET 8: `System.Linq.Expressions` y `System.Reflection` para análisis de expresiones.

24.2.2 Convenciones

`ForEachAsync` es secuencial; si necesitas concurrencia controlada, considera patrones adicionales (p. ej., `Parallel.ForEachAsync` o pipelines).

24.3 EJEMPLOS DE USO Y CASOS DE USO

Ejemplo: `ForEach` y `ForEachAsync`

```
// C#
using Framework.Utills.Extesions;

var items = new[] { 1, 2, 3 };
items.ForEach(i => Console.WriteLine(i));

await items.ForEachAsync(async i =>
{
    await Task.Delay(10);
    Console.WriteLine(i);
});
```

Ejemplo: Obtener nombre de propiedad desde una expresión

```
// C#
using Framework.Utills.Extesions;

public class Persona { public string Nombre { get; set; } = ""; }

string prop = ExpressionExtesnsions.GetPropertyNames<Persona>(p =>
p.Nombre);
// prop == "Nombre"
```

24.4 CASOS DE USO TÍPICOS

- Simplificar bucles sobre colecciones en servicios y handlers.

- Derivar nombres de propiedades para validaciones, mapeos, logs o generación de consultas.

24.5 BUENAS PRÁCTICAS

- Evita usar ForEach en colecciones extremadamente grandes con acciones costosas; evalúa rendimiento.
- Para operaciones paralelas, no uses ForEachAsync secuencial; opta por alternativas concurrentes.

24.6 VENTAJAS DE LA LIBRERÍA

- Productividad
 - Menos código repetitivo para operaciones comunes.
- Legibilidad
 - Expresividad al trabajar con colecciones y expresiones.
- Consistencia
 - Un único lugar para helpers compartidos entre módulos.

25 CONTROL DE ACCESO

☐ **Documento Público:** Información sin restricciones de reproducción, comunicación o transmisión completa o de cualquiera de sus partes interna o a terceros.

☒ **Documento Privado:** Información de uso interno. Disponible para todos los procesos internos autorizados. Esta información no puede ser conocida por terceros sin autorización formal y expresa.

☐ **Documento Confidencial:** Información privilegiada. Disponible solo para la prestación de los servicios o la operación del proceso responsable. No es permitida la reproducción, comunicación o transmisión sin la debida autorización formal y expresa del CEO, CTO.

Acceso	Observaciones
Líderes de área y equipo de arquitectura	<>

26 CONTROL DE VERSIONES

Versión	Fecha	Aprobó	Cargo	Notas
01	2025/09/14	Juan Diego Ramirez	Líder Técnico .Net	Creación